

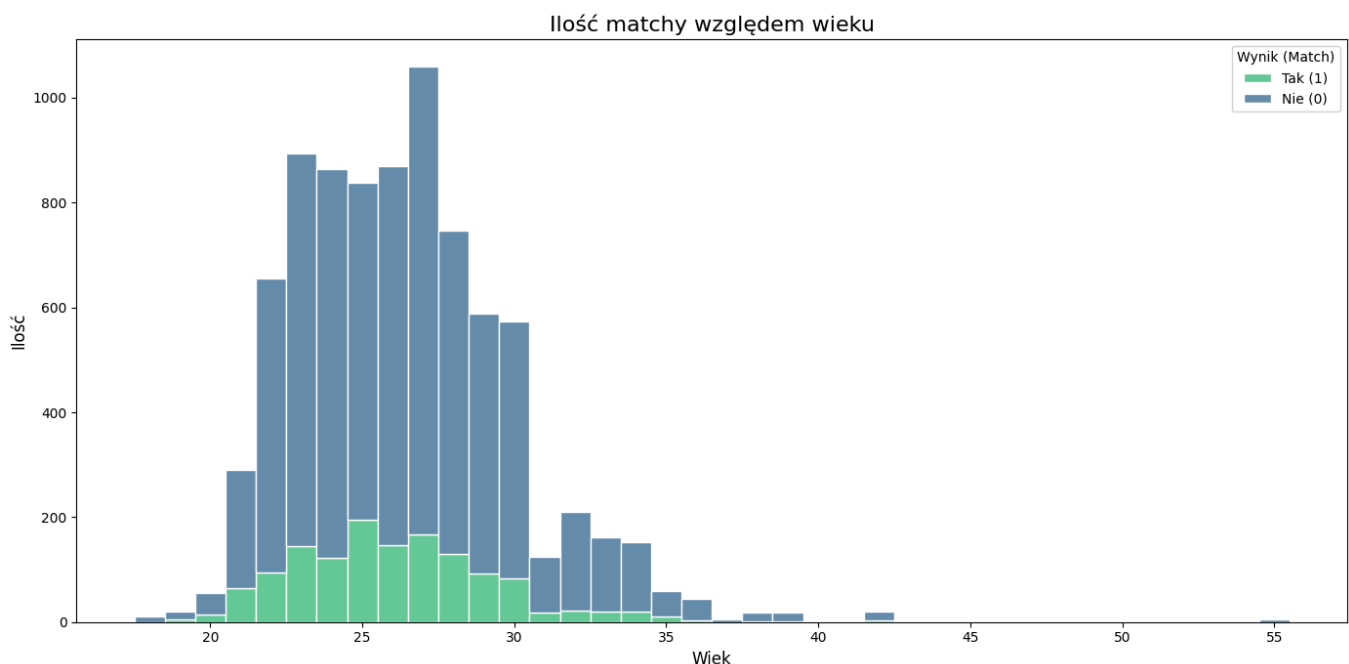
Laboratorium 8 - Piotr Jakóbczyk

Jako laboratorium do którego porównywane będą wyniki wybrałem laboratorium 6 dotyczące speed datingu (czyli działania na zbiorach nie zrównoważonych). Po przejrzaniu konkursów kaggle'a najbliższy mojemu zadaniu wydawał się zbiór [Give Me Some Credit](#), ponieważ też dotyczy on danych silnie niezbalansowanych i jego cechy są znane. Jako rozwiązania, do których porównywałem swoje rozwiązania przyjąłem następujące zgłoszenia:

- <https://www.kaggle.com/code/simonpfish/comp-stats-group-data-project-final>
- <https://www.kaggle.com/code/nicholasgah/eda-credit-scoring-top-100-on-leaderboard>
- <https://www.kaggle.com/code/prasadposture121/financial-distress-prediction>

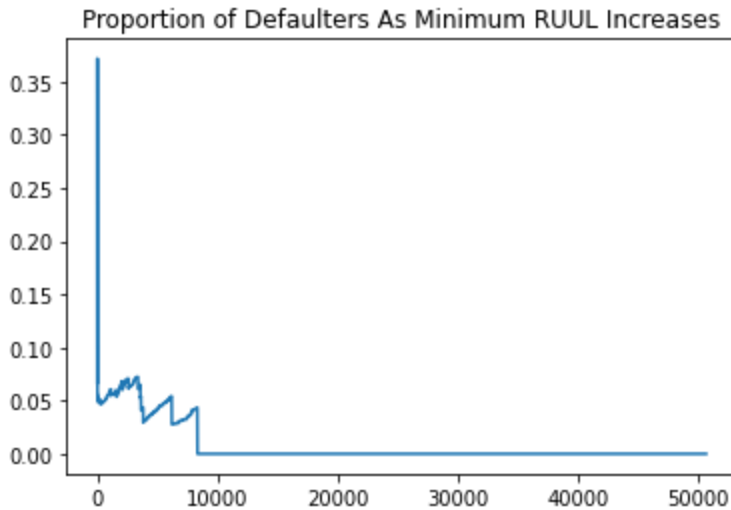
EDA

W moim notatniku EDA jest praktycznie nieistniejąca, wyświetlam tylko najbardziej podstawowe wykresy i zależności. Przydatne tutaj mogłoby być poszukanie czy nie ma jakichś korelacji nieliniowych. Dla przykładu, w moim notatniku pojawia się następujący wykres:



Wynika z niego że najwięcej matchy mają osoby w wieku 25 lat, natomiast nie jest to zbyt użyteczne, ponieważ ludzi w tym wieku jest bardzo dużo. Bardziej użyteczne mogłoby być tutaj

policzenie proporcji matchy dla każdego wieku, tak jak jest to robione np w eda - top 100 :



W tym samym pliku także sprawdzane są anomalie nie na poziomie jednej kolumny, tylko na poziomie kilku kolumn jednocześnie - ja tego nie robię, a mogłoby być przydatne poszukanie takich anomalii (np. osoby które wskazują yogę jako zainteresowanie ale ćwiczeń już nie):

```
distinct_triples_counts = dict()
for arr in df.loc[df["NumberOfTimes90DaysLate"] > 17][late_pay_cols].values:
    triple = ",".join(list(map(str, arr)))
    if triple not in distinct_triples_counts:
        distinct_triples_counts[triple] = 0
    else:
        distinct_triples_counts[triple] += 1
```

Preprocessing

W pliku comp-stats stosowana jest dogłębna analiza outlierów np.

```
df[df['DebtRatio'] > 3489.025]
[['SeriousDlqin2yrs', 'MonthlyIncome']].describe()
df[(df['DebtRatio'] > 3489.025) &
    (df['SeriousDlqin2yrs'] == df['MonthlyIncome'])].shape[0]
```

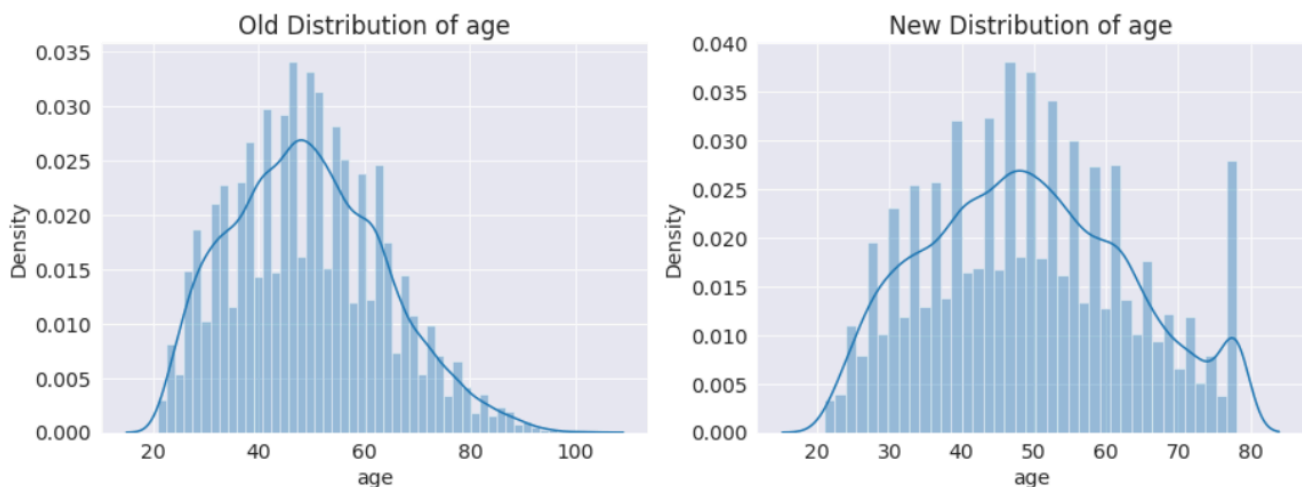
czyli sprawdzamy czy dla danych niestandardowych, ich proporcja innej cechy np. dla cechy oznaczającej klasę jest faktycznie inna. Dla przykładu - w moim datasetcie np jeśli dla kolumny expected_num_matches gdy jej wartość wynosi > 10, a match rate wtedy jest równy 0%, to jest to prawdopodobnie błędne wypełnienie ankiety i należy ten wiersz usunąć. Ta zmiana okazała się bardzo przydatna w oryginalnym datasetcie, sam autor zaznaczył: "However, there's a noticeable gain in performance between simply dropping the missing values and the modified datasets accross all models."

W pliku `financial-distress-prediction` kolumny, w których możliwe są duże odchylenia (np. wiek) są normalizowane w następujący sposób:

```
def normalizer(x, df):
    upper_boundary = df[x].mean() + 2 * df[x].std()
    lower_boundary = df[x].mean() - 2 * df[x].std()
    max_att = df[x].max()
    min_att = df[x].min()
    return {'Attribute': x, 'upper_boundary': upper_boundary,
            'lower_boundary': lower_boundary,
            'max_att': max_att, 'min_att': min_att}

def NormAtt(i, lim_df, df):
    Att = lim_df.iloc[i].Attribute
    UL = lim_df.iloc[i].upper_boundary
    LL = lim_df.iloc[i].lower_boundary
    fig, axes = plt.subplots(1, 2, figsize=(15, 5))
    axes[0].set_title('Old Distribution of ' + Att)
    sns.distplot(df[Att], ax=axes[0])
    df.loc[df[Att] < LL, Att] = LL
    df.loc[df[Att] > UL, Att] = UL
    axes[1].set_title('New Distribution of ' + Att)
    sns.distplot(df[Att], ax=axes[1])
```

czego celem jest zredukowanie ogonów (a są one długie np. w kolumnie wiek). Jest to bardzo agresywne podejście, ale daje dobre rezultaty, szczególnie dla drzew. Należałoby je przetestować w moim kodzie. Dla wieku tak wygląda przed/po:



Optimalizacja hiperparametrów

Ja po prostu odpalam GridSearch. Jak już skończy działać i da mi hiperparametry to nie jestem w stanie stwierdzić jak bardzo każdy z nich był ważny. `financial-distress` robi to następująco:

```
def test_params(**params):
    model = DecisionTreeClassifier(random_state=42, **params)
    model.fit(X_train, train_targets)
    train_score = accuracy_score(model.predict(X_train), train_targets)
    val_score = accuracy_score(model.predict(X_val), val_targets)
    return train_score, val_score

def test_param_and_plot(param_name, param_values):
    train_acc, val_acc = [], []
    for value in param_values:
        params = {param_name: value}
        train_score, val_score = test_params(**params)
        train_acc.append(train_score)
        val_acc.append(val_score)
    plt.figure(figsize=(10, 6))
    plt.title('Overfitting curve: ' + param_name)
    plt.plot(param_values, train_acc)
    plt.plot(param_values, val_acc)
    plt.xlabel(param_name)
    plt.ylabel('Accuracy')
    plt.legend(['Training', 'Validation'])
    print('Max Acc by:', val_acc.index(max(val_acc)) +
          int(min(param_values)))

def test_params(**params):
    model = RandomForestClassifier(random_state=42, n_jobs=-1, **params)
    ...

def test_params(**params):
    model = RandomForestClassifier(random_state=42, n_jobs=-1,
                                   n_estimators=800, **params)
    ...

def test_params(**params):
    model = RandomForestClassifier(random_state=42, n_jobs=-1,
                                   n_estimators=800, max_depth=12, **params)
    ...
```

Nie jest to w pełni optymalne podejście - m.in. nie robi ono CV. Prawdopodobnie najlepszym podejściem byłoby nie rezygnowanie z GridSearch, i spojrzenie na atrybut `results_`, aby móc

się dowiedzieć czegoś więcej o tym który hiperparametr okazał się najważniejszy i może dalsze eksperymentowanie z nim

Dodatkowy błąd - w swoim rozwiązaniu dziele wiersze na grupy względem "odcisku cyfrowego" użytkownika, ale tych grup nie używam w GridSearch. Powinienem był użyć GroupKFold, albo nawet wariantu stratyfikowanego (choć dla drzew stratyfikacja nie ma aż tak dużego znaczenia)

Uczenie modelu

W moim kodzie za każdym razem używam inżynierii cech w Pipeline, nawet nie wiedząc czy jest to faktycznie optymalne. W podanych notatnikach testowane jest po kilka wariantów transformacji danych, a nie tylko jeden.

Moje metody bazują na SMOTE (zgodnie z poleceniem). financial-distress używa bardzo agresywnego undersamplingu. prawdopodobnie należałoby się z tym bardziej pobawić:

```
train0 = train_df[train_df['SeriousDlqin2yrs'] == 0].sample(frac=0.06684)
train1 = train_df[train_df['SeriousDlqin2yrs'] == 1].copy()
train_df = pd.concat([train0, train1], axis=0)
train_df['SeriousDlqin2yrs'].value_counts()
```

Ewaluacja

W financial-distress dla lasu losowego testowane jest każde pojedyncze poddrzewo. Mogłoby być to pożyteczne, ponieważ w praktyce może się okazać że n_estimators=150 daje bardzo podobne wyniki do tego gdybyś wzięli po prostu 5 najlepszych z niego poddrzew:

```
def estimator_acc(i):
    model.estimators_[i].fit(X_train, train_targets)
    return model.estimators_[i].score(X_val, val_targets)
accuracy = []
for i in range(0, 100):
    accuracy.append(estimator_acc(i))
max(accuracy)
```

Podczas mierzenia metryk, nie robię tego także używając walidacji krzyżowej (wewnętrznie tak to się robi przez GridSearch, ale nie pokazuje tego):

```
s = cross_validate(clf, X, Y, scoring=['roc_auc'], cv=cv, n_jobs=-1)
self.cache[(m_name, df_name, sample_len, cv)] = s

for score in sorted(scores.items(),
```

```
key=lambda x: -1 * x[1]['test_roc_auc'].mean())[:10]:
auc = score[1]['test_roc_auc']
print("%s --> AUC: %0.4f (+/- %0.4f)" %
      (str(score[0]), auc.mean(), auc.std()))
```